LIBRARY MANAGEMENT SYSTEM REPORT

Title Page

Course Name: Object-Oriented Programming (OOP) – Term II
Project Title: Library Management System
Student Name: *Tanisha Chakrabarti*
Roll Number: *12*
Instructor Name: *Soumadip Biswas*
Session: Spring 2026

Problem Statement

In today's digital environment, managing library resources manually is inefficient and prone to errors. Traditional systems make it difficult to track books, manage members, and handle issuing and returning operations effectively.

The objective of this project is to develop a **Library Management System using Java** that allows users to add, update, delete, and search books, manage members, and perform operations like issuing and returning books. The system ensures proper validation to prevent invalid actions such as issuing unavailable books or returning non-issued books.

This project demonstrates the use of **Object-Oriented Programming (OOP) concepts** such as encapsulation, abstraction, and polymorphism. It also follows a structured design approach to improve maintainability and scalability.

System Design

UML Class Diagram (Text Representation)

```
+----------------------+
|          Book        |
+----------------------+
| - id: int            |
| - title: String      |
| - author: String     |
| -isIssued: boolean|
+----------------------+
| + issueBook()        |
| + returnBook()       |
| + display ()         |
+----------+-----------+
+----------------------+
|       Member         |
+----------------------+
| - id: int            |
| - name: String       |
+----------------------+
| + display ()         |
+----------------------+
+-----------------------+
```

```
|          Library          |
+-----------------------+
| - books: ArrayList    |
| -members:ArrayList|
+-----------------------+
| + addBook()           |
| + updateBook()        |
| + deleteBook()        |
| + searchBook()        |
| + issueBook()         |
| + returnBook()        |
 | + showStats()        |
+-----------------------+
```

Description of Major Classes

Book Class

- Stores book details such as ID, title, and author
- Tracks whether a book is issued

Member Class

- Stores member details
- Represents library users

Library Class

- Contains all core logic
- Handles book operations, member management, and statistics

Main Class

- Provides menu-driven interface
- Interacts with use

Package Structure

```
src/
└── com/
    └── library/
        └── main/
            ├── Book.java
            ├── Member.java
            ├── Library.java
            └── Main.java
```

Interaction Flow

1. User starts the application
2. Menu is displayed
3. User selects an operation
4. Library class processes the request
5. Results are displayed

OOP Concepts Demonstrated

Encapsulation

Where Used:
Book and Member classes
Why Used:
To protect data and control access
Example:

```
private String title;
public String getTitle() {
    return title;
}
```

Abstraction

Where Used:
Library class
Why Used:
To hide internal implementation
Example:

```
public void addBook(Book b) {
    books.add(b);
}
```

Inheritance

Where Used:
(Can be extended for exception handling)

Polymorphism

Where Used:

Methods performing similar operations

Why Used:

Improves flexibility

Design Pattern

Pattern Used:

Simple Layered Approach

Why Used:

Separates user interface and logic

Implementation Highlights

Book Management

- Add, update, delete books
- Search by title, author, ID

Issue & Return System

```
if (!b.isIssued()) {
    b.issueBook();
}
```

Justification:

Ensures only available books are issued

Member Management

- Add new members
- Display all members

Input Validation

```
if (id <= 0) {
    System.out.println("Invalid ID");
}
```

Justification:

Prevents invalid data entry

Testing & Error Handling

Test Cases

- Add book
- Search book
- Issue and return book
- Add member

Edge Cases

- Issuing already issued book
- Returning non-issued book
- Searching non-existing book

Failure Scenarios

- Invalid ID → Error message
- Book not found → Notification
- Empty list → Proper handling
-

Git Workflow

Git was used to:

- Track changes
- Maintain version history
- Create multiple commits
- Upload project to GitHub

Conclusion & Future Scope

Conclusion

The Library Management System successfully demonstrates the use of Java and OOP concepts in building a real-world application. The system is user-friendly, modular, and efficient.

Future Scope

- Add database integration (MySQL)
- Develop GUI using Java Swing/JavaFX
- Add login system
- Implement fine calculation

- Convert into web/mobile application